

Fenix FEM

Plataforma de Análisis por Elementos Finitos en Python

Manual de Usuario y Guía de Referencia

Documentación Oficial y Arquitectura del Sistema

Versión 1.0

Autor: Jaime Retama Velasco

Facultad de Estudios Superiores Aragón
Universidad Nacional Autónoma de México

12 de abril de 2026

Índice general

Resumen Ejecutivo	III
1. Introducción y Configuración del Entorno	1
1.1. Filosofía del Proyecto	1
1.2. Requisitos del Sistema	1
1.2.1. Entorno Base	1
1.2.2. Instalación de Dependencias Core	1
1.3. Herramientas Externas Recomendadas	2
2. Archivo de Entrada YAML	3
2.1. Flujo de Trabajo y Ejecución	3
2.2. Estructura del Documento YAML	3
2.2.1. Importación de Mallas Externas (mesh)	4
2.2.2. Catálogo de Materiales (materials)	4
2.2.3. Imposición de Condiciones Cinemáticas y Cargas	5
2.2.4. Configuración del Motor Matemático (solver)	5
2.2.5. Directivas de Exportación Científica (output)	5
3. Librería de Elementos y Modelos Constitutivos	7
3.1. Elementos Finitos Unidimensionales	7
3.1.1. Barra Articulada (Truss2D / Truss3D)	7
3.1.2. Marco Rígido Bidimensional (Frame2DEuler / Frame2DTimoshenko)	8
3.2. Elementos Finitos Bidimensionales	8
3.2.1. Cuadrilátero Bilineal (Quad4)	8
3.2.2. Triángulo de Deformación Constante (Tri3)	9
3.3. Leyes Constitutivas y Criterios de Falla	9
3.3.1. Elasticidad Lineal Isótropa (Elastic2D)	9
3.3.2. Elastoplasticidad de Von Mises (VonMises2D)	10
3.3.3. Daño Isótropo Continuo (IsotropicDamage2D)	10
4. Post-Procesamiento y Visualización VTK	11
4.1. El Formato XML de Kitware (VTU)	11
4.1.1. Datos Nodales (<i>Point Data</i>)	11
4.1.2. Datos de Celda (<i>Cell Data</i>)	11
4.2. Flujo Operativo en ParaView	12

5. Ejemplos Completos y Casos de Estudio	13
5.1. Placa con Perforación Central a Tracción	13
6. Uso Avanzado: API Programática (Python)	15
6.1. El Patrón Fachada (<i>Facade</i>)	15
7. Diagnóstico y Resolución de Problemas (Troubleshooting)	17

Resumen Ejecutivo

Fenix FEM representa una solución computacional avanzada y altamente optimizada para el Análisis por Elementos Finitos (FEA), concebida para abordar los desafíos más exigentes de la mecánica de sólidos bidimensional, la evaluación continua del daño estructural y el comportamiento elastoplástico de los materiales modernos. Desarrollado con una arquitectura orientada a la eficiencia y la escalabilidad, Fenix FEM se posiciona como una herramienta vital tanto para la industria del diseño estructural como para la investigación aplicada.

A diferencia de los paquetes comerciales cerrados, Fenix FEM ofrece total transparencia en sus formulaciones, implementando rigurosamente algoritmos de vanguardia como *Return Mapping* para plasticidad, control adaptativo de pasos de carga, solucionadores robustos de Newton-Raphson y trazado de curvas de equilibrio inestables mediante el método de Longitud de Arco (*Arc-Length* de Crisfield).

El núcleo del sistema ha sido diseñado bajo una filosofía *Data-Driven* (Impulsada por Datos), eliminando la curva de aprendizaje asociada a la programación o a las complejas interfaces gráficas (GUI) propietarias. Toda la orquestación de la simulación, desde la importación de discretizaciones topológicas complejas hasta la exportación de tensores de esfuerzos, se gobierna mediante un **archivo de entrada YAML**.

Este manual de usuario y guía de referencia técnica ha sido elaborado para documentar exhaustivamente todas las capacidades operativas del motor. A través de sus páginas, el ingeniero o analista encontrará:

1. Los principios fundamentales para la configuración del entorno de simulación.
2. Un desglose riguroso de la sintaxis del archivo de entrada YAML soportada por el analizador semántico de Fenix FEM.
3. El catálogo completo de leyes constitutivas de materiales, abarcando desde la elasticidad lineal hasta la mecánica de fractura continua.
4. Procedimientos estandarizados para la post-visualización científica de alto rendimiento utilizando *ParaView*.
5. Casos de estudio completos y guías de resolución de problemas (Troubleshooting) para el tratamiento de matrices singulares o problemas de convergencia no lineal.

Capítulo 1

Introducción y Configuración del Entorno

1.1 Filosofía del Proyecto

El diseño estructural contemporáneo y la validación de componentes mecánicos demandan herramientas capaces de predecir con precisión no solo el comportamiento elástico lineal de los cuerpos, sino su ruta patológica de degradación, plastificación localizada y eventual colapso sistémico. Fenix FEM nace con el objetivo de cubrir esta brecha, proporcionando un entorno numérico estable y altamente paralelizable en rutinas críticas, apoyándose en las bibliotecas de alto rendimiento para el álgebra lineal tensorial. Al delegar la pre-configuración a archivos YAML legibles por humanos, Fenix FEM permite el versionado sencillo de los modelos estructurales en repositorios como Git, fomentando la colaboración entre equipos de ingeniería.

1.2 Requisitos del Sistema

Fenix FEM es multiplataforma y se ejecutará de forma nativa en Windows, distribuciones basadas en Linux y macOS. La arquitectura del motor no requiere de complejas compilaciones de código en C++ ni configuraciones de `Makefiles`, basándose enteramente en el ecosistema científico de Python.

1.2.1 Entorno Base

Deberá contar con **Python 3.8 o superior** instalado en su sistema. En sistemas operativos Windows, es estrictamente necesario marcar la casilla *"Add Python to PATH"* durante el proceso de instalación. Se recomienda encarecidamente la utilización de entornos virtuales (`venv` o `Conda`) para aislar las dependencias del motor del resto de las bibliotecas del sistema operativo, garantizando así la reproducibilidad absoluta de los cálculos.

1.2.2 Instalación de Dependencias Core

El núcleo matemático y las rutinas de traducción de Fenix FEM requieren bibliotecas específicas altamente optimizadas. Abra su terminal o símbolo del sistema e instale el entorno

ejecutando:

```
pip install numpy scipy pyyaml meshio
```

- **numpy y scipy:** Constituyen el corazón del solucionador algebraico. Se encargan del ensamblaje masivo y la inversión de matrices dispersas (*Sparse Matrices*) en formatos CSR/CSC para optimizar la memoria RAM.
- **pyyaml:** Responsable del análisis semántico (*parsing*) del archivo de entrada YAML de la interfaz principal.
- **meshio:** Puente traductor bidireccional. Permite interpretar topologías complejas desde Gmsh y exportar los tensores a formato VTK.

1.3 Herramientas Externas Recomendadas

Aunque Fenix FEM es computacionalmente autónomo, se apoya en software de terceros *Open Source* para las fases de pre y post-procesamiento visual.

- **Gmsh (Pre-procesador):** Generador de mallas por elementos finitos avanzado. Se recomienda utilizarlo para modelar geometrías 2D continuas (placas con perforaciones, muros de corte) y exportar la discretización en formato `.msh`.
- **ParaView (Post-procesador):** Aplicación de análisis de datos y visualización líder en la industria. Fenix FEM exporta nativamente los campos de resultados en formato `.vtu` (VTK XML) para su posterior renderizado y análisis interactivo.

Capítulo 2

Archivo de Entrada YAML

La interacción exclusiva y oficial con el motor de Fenix FEM se realiza a través del archivo de entrada `.yaml`. Este enfoque elimina drásticamente los errores de sintaxis asociados a la programación mediante *scripts*, asegurando que el ingeniero se concentre puramente en la física del problema y no en la lógica del código subyacente.

2.1 Flujo de Trabajo y Ejecución

El flujo operativo estándar de Fenix FEM consta de tres fases claramente delimitadas:

1. **Pre-procesamiento (Geometría):** Modelado del dominio físico continuo en el software externo Gmsh, asignando grupos físicos (*Physical Groups*) a las superficies y fronteras críticas. Exportación de la malla en formato `.msh`.
2. **Configuración (YAML):** Redacción del manifiesto `.yaml` que vincula la malla con las propiedades constitutivas, las restricciones cinemáticas, las cargas y los parámetros del solucionador matemático.
3. **Análisis y Post-procesamiento:** Procesamiento del manifiesto por el motor analítico de Fenix FEM y subsecuente interpretación visual de los archivos VTU generados.

Para iniciar el análisis computacional, diríjase a la raíz del proyecto mediante su consola de comandos y ejecute el analizador apuntando a su archivo de configuración:

```
python fenix/utils/yaml_parser.py ruta/hacia/su_modelo.yaml
```

2.2 Estructura del Documento YAML

Un archivo YAML válido para Fenix FEM está compuesto por bloques estrictamente definidos. A continuación, se presenta un desglose exhaustivo de la gramática requerida.

2.2.1 Importación de Mallas Externas (mesh)

El bloque `mesh` es el punto de entrada de la geometría. Fenix FEM soporta la ingestión de mallas compuestas por elementos cuadriláteros bilineales (`Quad4`) y triángulos de deformación constante (`Tri3`).

```
mesh: "geometria_placa.msh"
mesh_material: 1
mesh_thickness: 0.1
mesh_quadrature: "2x2"
```

Parámetro	Descripción
<code>mesh</code>	Ruta absoluta o relativa hacia el archivo de malla <code>.msh</code> exportado por Gmsh.
<code>mesh_material</code>	Número entero (ID) que enlaza a toda la malla con un material del catálogo de propiedades.
<code>mesh_thickness</code>	Espesor de la lámina bidimensional (en metros) para el cálculo de rigidez volumétrica.
<code>mesh_quadrature</code>	Regla de integración de Gauss. "2x2" (Completa) o "1x1" (Reducida). Se recomienda siempre "2x2" para evitar el efecto "Hourglassing".

Tabla 2.1: Parámetros globales del bloque de Malla.

Mapecto Avanzado de Grupos Físicos (mesh_physical_groups)

En problemas de la vida real, una estructura rara vez es monolítica. Si su geometría en Gmsh posee zonas con diferentes composiciones (por ejemplo, un muro de hormigón armado con incrustaciones metálicas, o zonas de distinto espesor), Fenix FEM permite realizar un mapeo explícito de los nombres asignados en el CAD hacia las propiedades mecánicas internas.

```
mesh_physical_groups:
  "Zona_Hormigon":
    material: 1
    thickness: 0.20
    quadrature: "2x2"
  "Zona_Acero":
    material: 2
    thickness: 0.05
```

2.2.2 Catálogo de Materiales (materials)

La definición de las propiedades mecánicas y leyes constitutivas es el núcleo físico de la simulación. Dada su vital importancia paramétrica, la documentación exhaustiva de este bloque, junto con los tipos de elementos geométricos soportados por el motor, se detalla a profundidad en el **Capítulo 3: Librería de Elementos y Modelos Constitutivos**.

2.2.3 Imposición de Condiciones Cinemáticas y Cargas

Fenix FEM emplea un buscador espacial robusto para inyectar condiciones de Dirichlet (restricciones de soporte) y de Neumann (fuerzas) sobre el dominio sin que el usuario deba conocer los *IDs* numéricos de los nodos generados por Gmsh.

Búsqueda por Coordenadas (`_by_coord`)

Escanea la topología completa. Si la coordenada de un nodo coincide con el valor indicado (dentro del radio de tolerancia `tol`), se aplica la restricción.

```
boundary_conditions_by_coord:
  x_min: {ux: 0.0, uy: 0.0, tol: 1.0e-5} # Empotrar lado izquierdo
  rodillos: {coord: 'y', val: 10.0, uy: 0.0, tol: 1.0e-4}

point_loads_by_coord:
  carga_central: {coord: 'x', val: 5.0, uy: -15000.0, tol: 1e-4}
```

Búsqueda Topológica Inteligente (`_by_group`)

Constituye la metodología de vinculación más rigurosa para geometrías complejas. Requiere la definición previa de entidades físicas (*Physical Groups*) de tipo *Curve* (Líneas 1D) o *Point* (Nodos 0D) en las fronteras del dominio desde *Gmsh*, asignándoles un identificador semántico.

```
boundary_conditions_by_group:
  "Borde_Empotrado": {ux: 0.0, uy: 0.0}

point_loads_by_group:
  "Borde_Superior_Carga": {uy: -1500.0}
```

2.2.4 Configuración del Motor Matemático (`solver`)

La selección del algoritmo matricial dictamina la estabilidad de la simulación.

```
solver:
  type: NonlinearSolver
  num_steps: 20
  max_iter: 15
  tol: 1.0e-5
  adaptive: true
```

2.2.5 Directivas de Exportación Científica (`output`)

Instruye al motor sobre cómo generar la telemetría `.vtu` compatible con el ecosistema de visualización de Kitware (ParaView).

```
output:
  file_name: "resultados_placa"
  pre_process: {export: true} # Genera un archivo VTU del paso 0 (sin
                             deformar)
```

Comando	Efecto en el Ensamblador
LinearSolver	Invierte la matriz en un solo paso matemático. Fallará o dará resultados inconsistentes si se asocia a materiales como VonMises2D.
NonlinearSolver	Utiliza el método de Newton-Raphson. Particiona el vector de cargas externas por el número de pasos definidos en <code>num_steps</code> .
<code>adaptive: true</code>	Si la matriz tangente se vuelve singular o supera <code>max_iter</code> sin converger (Tolerancia del residuo <code>> tol</code>), el motor aborta, divide el paso de carga a la mitad (bisección), e intenta de nuevo.

```
results:
  frequency: "all_steps"          # Solo valido para NonlinearSolver (
    Exporta cada paso)
  nodal_results: ["Displacements"]
  element_results: ["Von_Mises", "Internal_State", "Sigma_XX"]
```

Capítulo 3

Librería de Elementos y Modelos Constitutivos

El motor de Fenix FEM posee una arquitectura modular que separa la formulación topológica espacial (el Elemento Finito) de la formulación física reológica (el Material). Esta sección detalla, desde un enfoque estrictamente práctico y orientado al usuario, los componentes disponibles para modelar sus estructuras y cómo parametrizarlos.

3.1 Elementos Finitos Unidimensionales

Aunque el enfoque principal de la importación de topologías radica en la mecánica de medios continuos 2D, el motor de Fenix FEM dispone de formulaciones reticulares unidimensionales de alto rendimiento, orientadas al modelado de celosías estructurales y sistemas de arriostramiento.

3.1.1 Barra Articulada (Truss2D / Truss3D)

Elemento estructural formulado para soportar exclusivamente cargas axiales (tracción y compresión directas). Sus nodos están teóricamente articulados, por lo que el elemento no posee rigidez a la flexión, y por ende, no transmite momentos flectores ni fuerzas cortantes.

- **Grados de Libertad (DOFs):** Cada nodo posee dos traslaciones (u_x, u_y) en el espacio bidimensional, o tres traslaciones (u_x, u_y, u_z) en el formulismo espacial 3D.
- **Casos de Uso:** Modelado de armaduras de cubiertas, puentes reticulares, torres de transmisión, o para actuar como cables tensores y refuerzos embebidos dentro de matrices de hormigón continuas.
- **Propiedades Geométricas Requeridas:** A diferencia de los elementos de área (Quad4) que requieren la definición de un espesor transversal (*thickness*), los elementos **Truss** exigen la definición del Área de la sección transversal (A) para computar su rigidez axial fundamental ($K = \frac{EA}{L}$).

Nota

Para incorporar elementos de armadura en el archivo de entrada YAML, es preciso definir las geometrías lineales (*Curves*) en el pre-procesador *Gmsh*, agruparlas bajo un *Physical Group* específico y adaptar la rutina de lectura en `yaml_parser.py` para la asignación de la sección transversal respectiva (*A*).

3.1.2 Marco Rígido Bidimensional (Frame2DEuler / Frame2DTimoshenko)

A diferencia de las armaduras, los elementos tipo *Frame* (Viga/Columna) poseen rigidez a la flexión y son capaces de transmitir momentos flectores y fuerzas cortantes transversales.

- **Grados de Libertad (DOFs):** Incorporan un tercer grado de libertad por nodo correspondiente a la rotación en el plano: (u_x, u_y, r_z) .
- **Variantes Teóricas:**
 - **Frame2DEuler:** Basado en la teoría de vigas esbeltas de Euler-Bernoulli. Ignora las deformaciones por cortante.
 - **Frame2DTimoshenko:** Orientado a vigas peraltadas o gruesas. Relaja la asunción de perpendicularidad de las secciones planas e incorpora la deformación por cortante al cálculo de la rigidez.
- **Propiedades Geométricas Requeridas:** En su configuración YAML, además del Área (*A*), el elemento **Frame2DEuler** requiere el Momento de Inercia (*I*). El elemento **Frame2DTimoshenko** requiere adicionalmente el Área Efectiva de Cortante (*As*).

```
elements:
- id: 1
  type: Frame2DTimoshenko
  A: 0.015          # Area transversal
  I: 1.25e-5        # Momento de Inercia
  As: 0.012         # Area de Cortante
```

3.2 Elementos Finitos Bidimensionales

La lectura e importación de archivos procedentes de *Gmsh* instanciará automáticamente los elementos correspondientes de acuerdo a la discretización topológica detectada. Aunque la declaración manual no es requerida, el dominio analítico de su comportamiento es fundamental para garantizar la convergencia matemática del modelo.

3.2.1 Cuadrilátero Bilineal (Quad4)

Es el elemento base tridimensional extruido y el más recomendado para cualquier simulación en el plano. Interpolado mediante funciones de forma bilineales, ofrece una excelente precisión en la captura de gradientes de esfuerzos en la flexión de cuerpos continuos.

- **Integración Numérica:** Utiliza la Cuadratura de Gauss. En su archivo YAML (bloque `mesh_quadrature`), se aconseja **siempre utilizar la regla completa "2x2"** (4 puntos de integración espaciales por elemento).
- **Advertencia de Modelado:** La adopción de una regla de integración reducida "1x1" disminuye la exigencia de memoria RAM en un 75 %; sin embargo, incrementa drásticamente la susceptibilidad de la estructura al fenómeno de *Hourglassing* (modos de deformación espurios de energía nula), comprometiendo severamente la validez del campo de esfuerzos obtenido.

3.2.2 Triángulo de Deformación Constante (Tri3)

Elemento de 3 nodos generado automáticamente por *Gmsh* en aquellas zonas donde la topología es demasiado compleja o angulosa para ser estructurada en cuadriláteros perfectos.

- **Comportamiento Físico:** Al poseer un único punto de integración implícito, asume que el esfuerzo calculado en todo su interior es constante. Esto lo vuelve un elemento matemáticamente muy rígido.
- **Recomendación de Mallado:** Una discretización predominante con este tipo de elemento induce el fenómeno de *Shear Locking* (bloqueo por cortante), derivando en una sobreestimación teórica de la rigidez global. En la fase de pre-procesamiento, se prescribe emplear el algoritmo **Recombine All** en *Gmsh* para priorizar la generación de cuadriláteros.

3.3 Leyes Constitutivas y Criterios de Falla

Los materiales se definen como una lista de diccionarios en el bloque `materials`. Cada material requiere un identificador numérico único (`id`) y un comportamiento dictado por la directiva `type`.

3.3.1 Elasticidad Lineal Isótropa (Elastic2D)

Modelo regido estrictamente por la Ley de Hooke Generalizada. Asume comportamiento perfectamente proporcional, donde el material recupera instantáneamente su forma original al retirar las fuerzas externas.

- **Casos de Uso:** Validación general de esfuerzos en régimen de pequeñas deformaciones, cálculo de rigideces elásticas iniciales, o modelado de acero/concreto trabajando de forma segura bajo su límite elástico de proporcionalidad.

```
materials:
- id: 1
  type: Elastic2D
  E: 200.0e9          # Modulo de Young o de Elasticidad (Pa)
  nu: 0.3             # Relacion de Poisson
  hypothesis: plane_stress
```

(Nota de parámetros: La propiedad `hypothesis` admite `plane_stress` (Tensión Plana) para simular placas delgadas libres de confinamiento en el eje Z; o `plane_strain` (Deformación Plana) para secciones transversales de presas o túneles infinitamente largos en el eje longitudinal).

3.3.2 Elastoplasticidad de Von Mises (VonMises2D)

Este modelo computacional evalúa el criterio de fluencia invariante J2. Simula la deformación plástica permanente de los materiales mediante algoritmos de *Return Mapping* cuando el estado de esfuerzos excede el parámetro σ_y .

- **Casos de Uso:** Es el modelo por excelencia en la industria metalmeccánica. Ideal para aleaciones dúctiles, acero estructural y aluminio trabajando en regímenes post-fluencia y de grandes disipaciones energéticas.

```
materials:
- id: 2
  type: VonMises2D
  E: 210.0e9
  nu: 0.3
  sigma_y: 250.0e6      # Limite Elastico / Esfuerzo de Fluencia (Pa)
  H: 2.0e9              # Modulo de endurecimiento plastico (Hardening)
```

3.3.3 Daño Isótropo Continuo (IsotropicDamage2D)

A diferencia de la plasticidad (que conserva la rigidez del cuerpo pero genera un cambio de forma permanente), el modelo de Daño Mecánico simula la aparición interna de microfisuras estructurales. Estas anomalías degradan paulatina e irremediabilmente el módulo de elasticidad del material (E), llevándolo eventualmente a un estado de fractura total (rigidez tendiendo a cero).

- **Casos de Uso:** Modelado constitutivo de materiales frágiles y cuasi-frágiles, donde la nucleación y coalescencia de fisuras conforman el mecanismo de rotura predominante (ej. hormigón simple, mampostería, macizos rocosos y cerámicas).

```
materials:
- id: 3
  type: IsotropicDamage2D
  E: 30.0e9              # Modulo original del material virgen
  nu: 0.2
  kappa_0: 0.00015      # Limite de deformacion (Inicio de fisuracion)
  alpha: 800.0           # Tasa de ablandamiento (Softening de la curva)
```

(Nota de ingeniería: El uso de este material requiere habilitar el modelo modificando previamente el módulo `yaml_parser.py` como se indica en la documentación técnica interna).

Capítulo 4

Post-Procesamiento y Visualización VTK

El volumen masivo de datos producidos en cada paso de cuadratura hace que la inspección en consola sea inviable. Fenix FEM integra un transpilador estandarizado orientado a **ParaView**, la suite líder mundial en análisis avanzado de dinámica de fluidos y elementos finitos.

4.1 El Formato XML de Kitware (VTU)

Mediante el uso de la biblioteca dependiente `meshio`, Fenix FEM compila toda la topología alterada del sistema y la información de los nodos/elementos inyectándola en archivos estructurados de extensión `.vtu`.

La estructura del exportador clasifica la información en dos diccionarios principales dictados por el estándar VTK:

4.1.1 Datos Nodales (*Point Data*)

Los atributos continuos mapeados espacialmente sobre los vértices:

- **Displacements:** Campo vectorial tridimensional $(u_x, u_y, 0, 0)$ de los desplazamientos calculados en el paso de carga actual.
- **External_Forces:** Distribución nodal del vector de cargas ensamblado en el lado derecho de la ecuación.
- **Supports:** Vector indicador booleano. Exhibe 1.0 en los grados de libertad que han sido coaccionados por empotramientos.

4.1.2 Datos de Celda (*Cell Data*)

Maape información promediada aritméticamente sobre el dominio interior de los elementos Finitos (sobre sus puntos de Gauss).

- **Von_Mises:** Cuantía escalar del criterio de fluencia invariante J2.

- **Internal_State:** El campo principal para rastrear el progreso de falla. Para plasticidad, captura la deformación plástica equivalente acumulada (α). Para daño continuo, muestra la severidad del agrietamiento escalar $d \in [0, 1]$.
- **Sigma_XX, Sigma_YY, Tau_XY:** Descomposición tensorial plana pura.

4.2 Flujo Operativo en ParaView

Para una experiencia visual de alto nivel, siga estrictamente estos pasos:

1. **Carga de la Colección:** Abra ParaView. Vaya a **File > Open** y seleccione el grupo de archivos colapsados (ej: `resultados_placa_..vtu`). Haga clic en *Apply* en el panel izquierdo (*Properties*).
2. **Animación Inicial:** Visualice los botones de reproducción de tiempo en la parte superior central. Al presionar "Play", ParaView iterará cronológicamente sobre todos los archivos de salida generados por el control de pasos.
3. **Deformación Realista (Warp By Vector):** Por defecto, ParaView renderiza la malla en su estado original $t = 0$, coloreando los esfuerzos sobre ella. Para que la geometría se distorsione físicamente, aplique el filtro **Warp By Vector**. Seleccione **Displacements** como variable y ajuste el "Scale Factor" según lo requiera (por ejemplo, escalar por 10x para visibilizar micro-deformaciones).
4. **Mapeo de Esfuerzos:** Seleccione el nodo final en la cascada de su filtro (*Pipeline Browser*). En la barra de herramientas, cambie **Solid Color** por **Von-Mises**. Active la visualización de bordes (*Surface With Edges*) para apreciar la distorsión de la matriz cuadrilátera de elementos **Quad4**.

Capítulo 5

Ejemplos Completos y Casos de Estudio

Para cimentar el flujo de trabajo guiado por datos, presentamos el modelado de un escenario clásico de concentración de tensiones.

5.1 Placa con Perforación Central a Tracción

Supongamos una placa de acero modelada en Gmsh (un cuadrado con un orificio circular en el medio). Hemos asignado el grupo físico "Superficie_Placa."^a los elementos 2D, el grupo "Borde_Fijo."^al lado izquierdo y "Borde_Carga."^al borde derecho.

El objetivo es superar el límite elástico del acero y observar el flujo de plasticidad alrededor del orificio.

```
mesh: "placa_agujero.msh"

mesh_physical_groups:
  "Superficie_Placa":
    material: 1
    thickness: 0.05
    quadrature: "2x2"

materials:
  - id: 1
    type: VonMises2D
    E: 200.0e9          # Acero A36
    nu: 0.3
    sigma_y: 250.0e6    # Fluencia a 250 MPa
    H: 1.5e9            # Endurecimiento cinemático

boundary_conditions_by_group:
  "Borde_Fijo": {ux: 0.0, uy: 0.0}

point_loads_by_group:
  "Borde_Carga": {ux: 1500000.0} # Tracción extrema

solver:
  type: NonlinearSolver
```

```
num_steps: 30          # Requerimos granularidad alta en la transicion a
    plasticidad
max_iter: 10
tol: 1.0e-5
adaptive: true         # Fundamental habilitarlo

output:
  file_name: "estudio_placa"
  pre_process: {export: true}
  results:
    frequency: "all_steps"
    nodal_results: ["Displacements"]
    element_results: ["Von_Mises", "Internal_State"]
```

Listing 5.1: Archivo YAML: Placa perforada elastoplástica

Interpretación de Resultados Esperados: El solucionador dividirá la carga masiva en 30 incrementos. Durante los primeros 10 pasos (asumiendo comportamiento elástico lineal), el *NonlinearSolver* convergerá casi instantáneamente en 1 a 2 iteraciones por paso. Cuando la concentración de esfuerzos de Von Mises en la periferia superior del orificio exceda los 250 MPa, el algoritmo calculará la matriz constitutiva elastoplástica de *Return Mapping*. Observará un aumento en las iteraciones requeridas de Newton-Raphson. En ParaView, la variable `Internal_State` exhibirá un mapa de calor donde el valor se aleja de cero, indicando las zonas de deformación permanente (plástica).

Capítulo 6

Uso Avanzado: API Programática (Python)

Aunque Fenix FEM fomenta el modelado guiado por datos (*Data-Driven*) mediante archivos YAML, su arquitectura subyacente de código limpio permite importarlo y utilizarlo directamente como una librería nativa de Python.

Esta metodología avanzada resulta indispensable para escenarios de ingeniería e investigación tales como:

- **Optimización Paramétrica:** Creación de bucles `for` para ejecutar docenas de simulaciones variando un parámetro (ej. el espesor de una placa) y trazar curvas de comportamiento óptimo.
- **Generación Procedural de Mallas:** Construcción de dominios geométricos complejos, fractales o repetitivos empleando funciones trigonométricas directamente en el código.
- **Inteligencia Artificial:** Acoplamiento del motor matemático a modelos de *Machine Learning* o algoritmos de evolución genética para encontrar formas topológicas óptimas.

6.1 El Patrón Fachada (*Facade*)

Para facilitar la legibilidad del código de los *scripts*, Fenix FEM expone todos sus componentes estructurales y físicos directamente desde la raíz del paquete. El usuario ya no requiere conocer la estructura interna de las carpetas.

```
import numpy as np
from fenix import Domain, VonMises2D, Quad4, Assembler, NonlinearSolver,
    VtkExporter

# 1. Definir el Dominio Físico y sus Nodos
domain = Domain()
n1 = domain.add_node(1, [0.0, 0.0])
n2 = domain.add_node(2, [1.0, 0.0])
n3 = domain.add_node(3, [1.0, 1.0])
n4 = domain.add_node(4, [0.0, 1.0])

# 2. Instanciar Material y Conectividad del Elemento
acero = VonMises2D(E=200e9, nu=0.3, sigma_y=250e6, H=2e9)
```

```
placa = Quad4(element_id=1, nodes=[n1, n2, n3, n4], material=acero,
              thickness=0.1)
domain.add_element(placa)

# 3. Restricciones Cinemáticas (Condiciones de Dirichlet)
n1.fix_dof('ux', 0.0); n1.fix_dof('uy', 0.0)
n4.fix_dof('ux', 0.0); n4.fix_dof('uy', 0.0)

# Indexar variables algebraicas
domain.generate_equation_numbers()

# 4. Cargas Externas (Condiciones de Neumann)
F_ext = np.zeros(domain.total_dofs)
F_ext[n2.dofs['ux']] = 150000.0 # Traccion nodal
F_ext[n3.dofs['ux']] = 150000.0

# 5. Orquestación y Solución del Sistema No Lineal
assembler = Assembler(domain)
solver = NonlinearSolver(assembler, num_steps=10, tol=1e-5, adaptive=True)

# Vector global de desplazamientos
U = solver.solve(F_ext)

# 6. Telemetría y Exportación Científica
exporter = VtkExporter(domain)
exporter.export("resultados_script.vtu", U=U, F_ext=F_ext)
```

Listing 6.1: Script estructural con la API programática de Fenix FEM

Capítulo 7

Diagnóstico y Resolución de Problemas (Troubleshooting)

El modelado estructural no lineal es matemáticamente complejo. A continuación se presentan las causas fundamentales de la interrupción en el orquestador de Fenix FEM y sus potenciales soluciones.

- **Excepción: *Matriz Singular Detectada en Predictor/Corrector***

Diagnóstico: El sistema no tiene la suficiente cantidad de restricciones cinemáticas impuestas. Se encuentra matemáticamente "flotando" en el espacio, permitiendo traslación de cuerpo rígido infinita.

Solución: Verifique el bloque `boundary_conditions_by_coord` o `_by_group`. Asegúrese de restringir al menos un nodo en U_x y U_y para anclar el dominio estáticamente.

- **Advertencia: *Bisección: reduciendo incremento... / El solucionador no convergió.***

Diagnóstico: El salto de carga provisto a la matriz tangente no lineal causó que los esfuerzos internos superasen catastróficamente la capacidad portante, arrojando residuos infinitos en el bucle Newton-Raphson.

Solución: Aumente `num_steps` en el bloque `solver`. Verifique que el umbral del material σ_y o κ_0 no esté mal tipeado (por ejemplo, escribir 250 en lugar de 250.0e6).

- **Error de Ingestión: *No se importó ningún elemento (Solid2D).***

Diagnóstico: El analizador `GmshParser` encontró el archivo, pero no localizó bloques de superficies (cuadriláteros o triángulos).

Solución: En Gmsh, asegúrese de haber creado una superficie bidimensional (*Plane Surface*) antes de generar la malla (2D Mesh). Fenix FEM filtra mallas espaciales para garantizar la consistencia topológica.